Teoría de Algoritmos
(TB024)
Guía de Ejercicios Prácticos

Universidad de Buenos Aires

Facultad de Ingeniería

Departamento de Computación

Curso Echevarría

Contenidos

Prólogo **4**
Bibliografía **5**
Bibliografía General 5
Bibliografía Complementaria 5
Capítulo 1 - División y Conquista **6**
Problema Resuelto 1.A 6
1.1) Ejercicios aplicando el Teorema Maestro 8
1.2) Arreglo, ¿ordenado o desordenado? 9
1.3) Encontrar el duplicado 9
1.4) Arreglo de unos y ceros 9
1.5) Parte entera de la Raíz Cuadrada 9
1.6) Encontrar el Pico 10
1.7) MergeSort Modificado 10
1.8) Producto de Matrices 10
1.9) Encontrar el elemento 10
Capítulo 2 - Algoritmos Greedy **12**
2.1) Envíos 1 - Mochila 12
2.2) Envíos 2 - Volumen 12
2.3) Envíos 3 - Mochila con volumen 12
2.4) Envíos 4 - Múltiples repartidores 12
2.5) Envíos 5 - Multivolumen 12
2.6) Convención 1 - Charlas 13
2.7) Convención 2 - Charlas con puntaje 13
2.8) Convención 3 - Charlas cortables 13
2.9) Convención 4 - Charlas cortables (cont.) 13
2.10) Arquitecto 1 - Cobertura de puertas 13
2.11) Arquitecto 2 - Cobertura de puertas (cont.) 14
2.12) Arquitecto 3 - Stands separados (discreto) 14
2.13) Arquitecto 4 - Stands separados (continuo) 14
2.14) Arquitecto 5 - Postas 14
2.15) Manager 1 - Guardia mínima 15
2.16) Manager 2 - Guardia mínima (cont.) 15
2.17) Manager 3 - Supervisores 15
2.18) Cambio 1 - Monedas 15
2.19) Cambio 2 - Trueque 15
2.20) Cambio 3 - Reforma monetaria (bonus) 16
Capítulo 3 - Backtracking **17**
3.1) Suma de Dados 17
3.2) Maximizar sumatoria a N 17
3.3) N Reinas 17
3.4) Sudoku 18

3.5) Knight Tour..... 18
3.6) Coloreo de Grafos..... 18
3.7) Camino Hamiltoniano..... 18
3.8) Maximum Independent Set..... 19
3.9) Isomorfismo de Grafos..... 19
3.10) Cuadrados de Lagrange..... 19
3.11) SEND + MORE = MONEY..... 19

Prólogo

Esta Guía de Ejercicios nació a pedido (o, más precisamente, por reclamo) de los estudiantes del curso de Teoría de Algoritmos. Durante varios cuatrimestres tomamos prestadas las guías de otras cátedras, hasta que finalmente juntamos coraje y decidimos construir la nuestra, con nuestra propia impronta y preferencias.

Como toda guía de ejercicios, no es un material exhaustivo; por el contrario, es una invitación a descubrir y despertar la curiosidad. Seguramente más de uno de ustedes lea un ejercicio y se quede con ganas de más: esa es nuestra intención como docentes.

Se dice por ahí que el que hace los ejercicios tiene mejores chances de aprobar el curso. La demostración empírica ya fue obtenida, no sólo en otras cátedras de la materia, sino en innumerables cursos y universidades a lo largo del mundo: existe una muy fuerte correlación entre practicar y aprobar. Como dice el credo del soldado espartano, “el que suda más en la práctica, sangra menos en la guerra”.

La pregunta natural que surge en estos días es si puedo usar una inteligencia artificial. La respuesta, naturalmente, es “claro que sí”, pero con cuidado y sin abusar. Si le pedimos que resuelva el problema, no nos sirve para formar el conocimiento dentro de nuestro cerebro. Recuerden que una inteligencia artificial no tiene creatividad: toma diversas fuentes y arma *algo* con eso, que la mayoría de las veces *parece* coherente: ese resultado necesita una verificación, un curado de su contenido. Pero en esta etapa de formación la recomendación es sentarse tranquilos frente al problema, con papel y lápiz (sí, con papel y lápiz, no con un teléfono, tablet o computadora), y empezar a jugar con los datos, patrones, ideas, darle vueltas hasta que se encienda la lámpara de la creatividad: esa chispa es la que nunca tendrá una inteligencia artificial, y es la que debemos entrenar en el estudio y la práctica. Es la chispa que nos permitirá reaccionar y pensar soluciones de cualquier tipo en la vida real. El día de mañana, cuando hayamos recorrido el camino del aprendizaje, sabremos con precisión cómo pedirle cosas a la inteligencia artificial, que podrá funcionar como una extensión natural de nuestra propia inteligencia.

Por último, dejamos unas palabras sobre los problemas resueltos. Éstos sirven para definir expectativas sobre lo que esperamos ver al enfrentar una situación problemática. Como mínimo, pretendemos que hagan un análisis para ver si el problema se parece a otro y si la forma de resolverlo se puede reutilizar total o parcialmente. Es un “pensar antes de actuar”. Luego, sí, un pseudocódigo: no pretendemos que sea código fuente listo para compilar, pero sí algo que pueda ser leído e interpretado unívocamente, sin ambigüedades, por otro ser humano. Y por último, un análisis de complejidad que sirva como autocrítica del algoritmo propuesto.

Bibliografía

Bibliografía General

- Kleinberg J., Tardos E. (2006). Algorithm Design, third edition. Cornell University.
- Cormen T. H., Leiserson C. E., Rivest R. L., Stein Clifford (2022). Introduction to Algorithms, fourth edition. The MIT Press.
- Sipser M. (2013). Introduction to the Theory of Computation, third edition. Cengage Learning.
- Goldreich, O. (2008). Computational Complexity, A conceptual perspective. Cambridge University Press.
- Taha, H (1997). Investigación de Operaciones, sexta edición. Prentice Hall.

Bibliografía Complementaria

- Echevarría, P (2004). El Método Simplex: Un enfoque práctico, segunda edición. https://campusgrado.fi.uba.ar/course/view.php?id=303&section=3
- Erickson, J. (2019). Algorithms. 1st Edition. http://jeffe.cs.illinois.edu/teaching/algorithms/
- Grimaldi, R. (2004). Discrete and Combinatorial Mathematics, fifth edition. Pearson Education.
- Hamming, R. (1997). The Art of doing Science and Engineering
- Karp, R. M. (1991). An introduction to randomized algorithms. Discrete Applied Mathematics 34, pag. 165-201.
- Martello, S y Toth, P (1990) Knapsack problems. John Wiley & Sons.
- Nilsson, Christian (2003?) Heuristics for The Traveling Salesman Problem.
- Wolsey, L. (1998). Integer Programming, second edition. John Wiley & Sons.

Capítulo 1 - División y Conquista

Problema Resuelto 1.A

Enunciado

Está asesorando a una pequeña empresa de inversiones que realiza simulaciones computacionalmente intensivas y necesita resolver repetidamente el siguiente problema.
Se consideran n días consecutivos de cotización de una acción, numerados $i=1, 2, ..., n$. Para cada día i se conoce el precio $p(i)$ por acción. Se desea comprar 1000 acciones en algún día i y venderlas todas en un día posterior j. El objetivo es determinar la máxima ganancia que puede obtenerse. Si durante esos n días no existe una operación con ganancia, debe informarse esta situación.

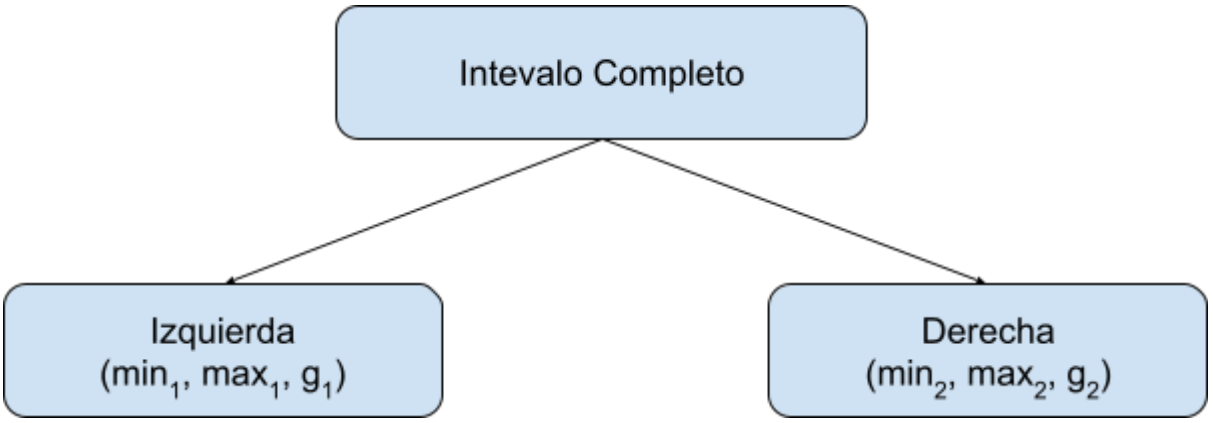
Ejemplo. Si $n=3$, $p(1)=9$, $p(2)=1$ y $p(3)=5$, la respuesta es comprar el día 2 y vender el día 3. La ganancia máxima es de $4 por acción.

Una solución directa consiste en probar todos los pares posibles de días de compra y venta. Este algoritmo tiene una complejidad $O(n^2)$. Se pide obtener una solución de mejor complejidad.

Análisis de la situación

<u>Caso recursivo general</u>

El padre combina exclusivamente lo que devuelven sus dos hijos.



Tenemos en cuenta las siguientes variables:

- min = min (min_1, min_2)
- max = max(max_1, max_2)
- g = max(g_1, g_2, max_2 - min_1)

Observación: el cruce usa max_2 - min_1 porque toda la rama izquierda ocurre antes que la derecha.

¿Qué devuelve cada llamada?
La función conserva la información necesaria para combinar ramas.

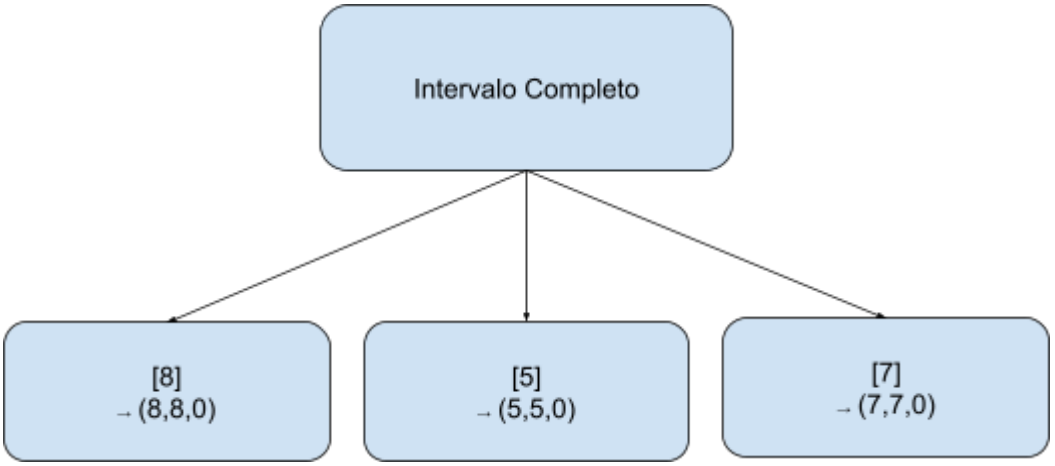
```
AnalizarIntervalo(precios, inicio, fin) → (mínimo, máximo, ganancia)
```

Donde:

- mínimo= menor precio del intervalo
- máximo: mayor precio del intervalo
- ganancia: mayor operación válida. Es decir, respeta el orden temporal: comprar antes de vender.

Caso Base

Dado el intervalo *[8,5,7]*, veamos cuáles serían los casos base:




```
si inicio = fin entonces
    devolver (precios[inicio], precios[inicio], 0)
fin si
```

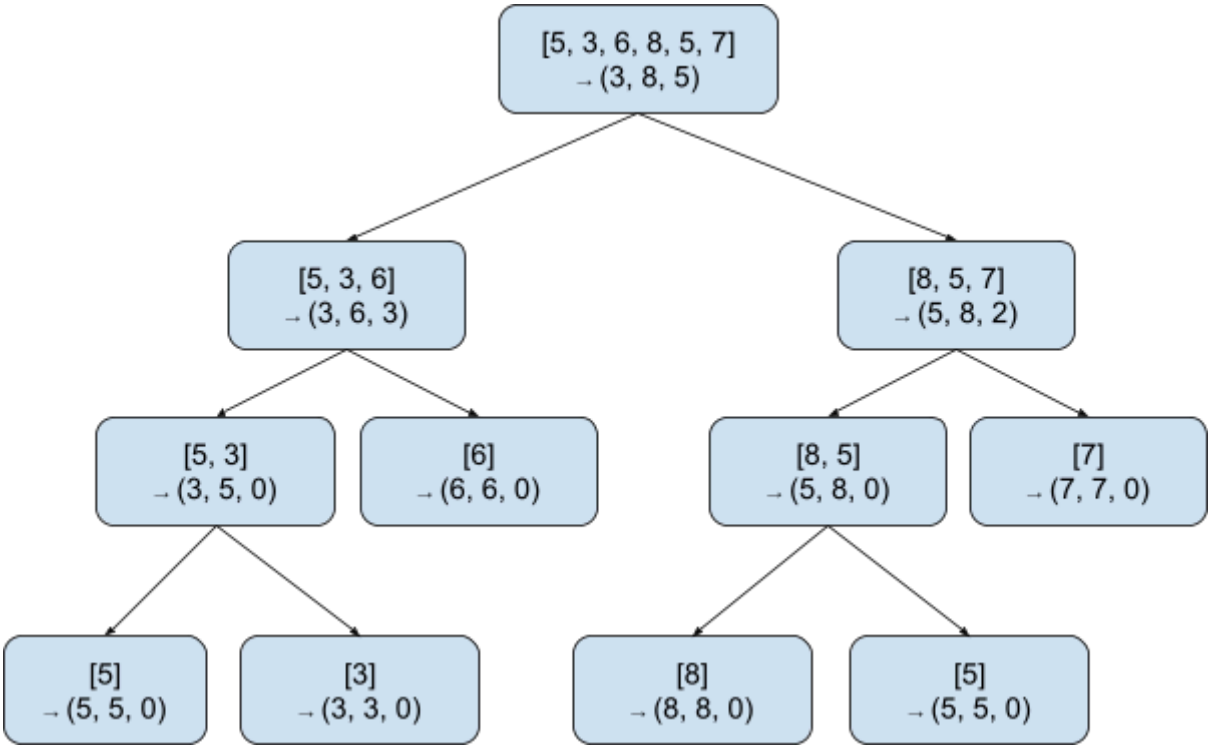
Observación: Con un único día no hay compra y venta posible, pero el mínimo y el máximo ya quedan determinados.

Pseudocódigo

```
AnalizarIntervalo(precios, inicio, fin)
    si inicio = fin entonces
        devolver (precios[inicio], precios[inicio], 0)
    fin si
    medio ← ⌊(inicio + fin) / 2⌋
    (min1, max1, g1) ← AnalizarIntervalo(precios, inicio, medio)
    (min2, max2, g2) ← AnalizarIntervalo(precios, medio + 1, fin)
    minimo ← min(min1, min2)
    maximo ← max(max1, max2)
    cruce ← max2 - min1
    ganancia ← max(g1, g2, cruce)
    devolver (minimo, maximo, ganancia)
```

Seguimiento del Pseudocódigo

Vamos a hacer un seguimiento del pseudocódigo sobre el intervalo *[5, 3, 6, 8, 5, 7]*.



La raíz compara:
ganancia izquierda = 3, ganancia derecha = 2, ganancia del cruce=8-3=5

Análisis de Complejidad

Ecuación de Recurrencia y Complejidad:

- $T(n)$: tiempo de ejecución sobre un intervalo con n elementos
- $2\,T\,(n/2)$: dos llamadas recursivas, cada una sobre la mitad del intervalo
- $\Theta(1)$: combinación con una cantidad constante de operaciones

Entonces:
$T(n) = 2\,T(n/2) + \Theta(1)$

Aplicando el Teorema Maestro:
$a=2,\ b=2, f(n)=\Theta(1)$
Y:
$n^a = n^2 = n$

Por lo tanto:
$T(n)=O(n)$

1.1) Ejercicios aplicando el Teorema Maestro

Indicar, utilizando el Teorema Maestro, la complejidad de cada una de las siguientes ecuaciones de recurrencia:

$A(n) = 2A(n/4) + \sqrt{n}$	$B(n) = 2B(n/4) + n$	$C(n) = 2C(n/4) + n^2$
$D(n) = 3D(n/3) + \sqrt{n}$	$E(n) = 3E(n/3) + n$	$F(n) = 3F(n/3) + n^2$
$G(n) = 4G(n/2) + \sqrt{n}$	$H(n) = 4H(n/2) + n$	$I(n) = 4I(n/2) + n^2$

1.2) Arreglo, ¿ordenado o desordenado?

Implementar, por división y conquista, una función que, dado un arreglo y su tamaño (cantidad de elementos), determine si el mismo se encuentra ordenado. Indicar y justificar el orden.

1.3) Encontrar el duplicado

Se cuenta con un arreglo de enteros ordenado de manera ascendente, que contiene exactamente un número duplicado (es decir, todos los demás elementos son distintos, sin duplicados). Implementar una función que encuentre dicho número utilizando división y conquista. Indicar y justificar la complejidad del algoritmo, utilizando el Teorema Maestro.

1.4) Arreglo de unos y ceros

Se tiene un arreglo tal que *[1,1,1,...,0,0,...]* (es decir, unos seguidos de ceros). Se pide:

a. Una función de orden $O(\log \log n)$ que encuentre el índice del primer 0. Si no hay ningún 0 (solo hay unos), debe devolver -1.
b. Demostrar con el Teorema Maestro que la función es, en efecto, $O(\log \log n)$.

Ejemplos:
[1, 1, 0, 0, 0] → *2*
[0, 0, 0, 0, 0] → *0*
[1, 1, 1, 1, 1] → *-1*

1.5) Parte entera de la Raíz Cuadrada

Implementar un algoritmo que, por división y conquista, permita obtener la parte entera de la raíz cuadrada de un número n, en tiempo $O(\log \log n)$. Por ejemplo, para $n=10$ debe devolver 3, y para $n=25$ debe devolver 5.

1.6) Encontrar el Pico

Se tiene un arreglo de $n>=3$ elementos en forma de pico, esto es: estrictamente creciente hasta una determinada posición p, y estrictamente decreciente a partir de ella (con $0<p<n-1$). Por ejemplo, en el arreglo *[1,2,3,1,0,-2]* la posición del pico es $p=2$. Se pide:

a. Implementar un algoritmo de división y conquista de orden $O(log\ log\ n)$ que encuentre la posición p del pico:

```
func PosicionPico(v []int, ini, fin int) int
```

La función será invocada inicialmente como: `PosicionPico(v, 0, len(v)-1)`, y tiene como precondición que el arreglo tenga forma de pico.

b. Justificar el orden del algoritmo mediante el Teorema Maestro.

1.7) MergeSort Modificado

Se quiere implementar MergeSort pero, en vez de dividir en dos partes el arreglo, dividirlo en tres, llamando recursivamente al algoritmo para cada una de las partes y luego uniéndolas.

a. Suponiendo que el merge de las tres partes se realiza en tiempo lineal, calcular el orden del algoritmo.
b. Si en vez de dividir en tres partes, se dividiera el arreglo en n, siendo n la cantidad de elementos del arreglo ¿a qué otro algoritmo de ordenamiento se asemeja esta implementación? ¿Cuál es el orden de dicho algoritmo?
c. Dado lo obtenido en los puntos anteriores ¿tiene sentido implementar MergeSort con k separaciones, para $k>2$?

1.8) Producto de Matrices

Un algoritmo sencillo para multiplicar matrices de $n\ x\ n$ demora $O(n^3)$. El algoritmo de Strassen (que utiliza División y Conquista) lo hace en $O(n^{2,8})$. La profesora Manterola quiere implementar un algoritmo de División y Conquista que sea aún más veloz, donde divida al problema en A subproblemas de tamaño de $n/4$, y que juntar las soluciones parciales sea $O(n^2)$. ¿Cuál es el máximo A para que el orden del algoritmo sea menor que el del algoritmo de Strassen? Justificar.

1.9) Encontrar el elemento

Implementar una función (que utilice división y conquista) de orden $O(n\ log\ log\ n)$ que, dado un arreglo de n números enteros, devuelva `true` o `false` según si existe algún elemento que aparezca más de la mitad de las veces. Justificar el orden de la solución. Ejemplos:

[1, 2, 1, 2, 3] -> `false`
[1, 1, 2, 3] -> `false`
[1, 2, 3, 1, 1, 1] -> `true`
[1] -> `true`

Aclaración: Este ejercicio puede resolverse, casi trivialmente, ordenando el arreglo con un algoritmo eficiente, o incluso se puede realizar más rápido utilizando una tabla de hash. Para hacer interesante el ejercicio, resolver *sin ordenar el arreglo*, empleando únicamente división y conquista. Por otro lado, también puede resolverse en $O(n)$ con división y conquista, pero resulta más complicado y lo dejamos para otro momento.

Capítulo 2 - Algoritmos Greedy

2.1) Envíos 1 - Mochila

Los hermanos Pill y Tipp montaron su empresa de envíos. Por viaje, su repartidora estrella puede llevar hasta cierto máximo de peso. Tienen cierta cantidad de objetos a enviar, y cada objeto tiene un cierto peso y un cierto valor monetario por ser entregado. Diseñar un algoritmo que le permita a los hermanos decidir qué objetos darle a su repartidor para maximizar las ganancias.

2.2) Envíos 2 - Volumen

Les llegó un pedido especial a los hermanos desde las montañas del norte. En este caso, la mercancía a entregar son jarras de pintura de laca que tienen un cierto volumen y peso, pero que valen todas lo mismo. Como la mercancía es delicada, su repartidora va a tener que utilizar un contenedor especial aislante que puede soportar hasta un máximo de peso y volumen. Diseñar un algoritmo que permita a los hermanos decidir cuáles jarrones enviar para maximizar la ganancia.

2.3) Envíos 3 - Mochila con volumen

Tras el último envío, la repartidora descubrió que es mucho mejor llevar paquetes de un tamaño no muy grande, y les exige ahora a los hermanos que no le asignen más de un volumen máximo de objetos en los envíos (además de la restricción de peso). Modificar el algoritmo para que los hermanos puedan asignar los objetos a enviar teniendo en cuenta su peso, volumen y valor (que pueden ser todos distintos).

2.4) Envíos 4 - Múltiples repartidores

El negocio de envíos está siendo exitoso, así que los hermanos decidieron contratar a más repartidores para poder hacer envíos más grandes. Para cada envío, le pueden asignar a cada repartidor hasta cierto peso que es el mismo para todos los repartidores (por ahora no les importa el volumen), y envían a cuantos repartidores hagan falta para enviar todos los items. Diseñar un algoritmo que divida los ítems en la mínima cantidad de repartidores posible.

2.5) Envíos 5 - Multivolumen

La repartidora estrella les contó a los nuevos sobre el máximo de volumen, y como les pareció una buena idea, decidieron entre todos armar un sindicato para exigir un máximo de volumen por repartidor (que también es el mismo para todos, fijado por el sindicato). Modificar el algoritmo para que realice la división de ítems en repartidores tomando en cuenta el peso y el volumen de cada ítem para minimizar la cantidad de repartidores.

2.6) Convención 1 - Charlas

Camila se encuentra en medio de la “Convención de Diseño Eficiente de Algoritmos 2027”. Como es muy fan de los algoritmos, le interesan todas las charlas por igual. Por política del centro cultural donde se organiza el evento, una vez empezada una charla no se vuelven a abrir las puertas hasta que finaliza, por lo que si quiere ir a una en particular tiene que estar desde el inicio hasta el fin de la misma. Tiene una lista con el horario de inicio y fin de cada charla. Diseñar un algoritmo que le permita a Carla organizar su agenda para asistir a la mayor cantidad de charlas posible.

2.7) Convención 2 - Charlas con puntaje

Diego también asistió a CDEA 2027, pero a diferencia de Camila, él es mucho más quisquilloso con las charlas a las que quiere asistir. Además de los horarios de inicio y fin, le asignó a cada charla un puntaje en función de qué tan interesante le parece. Diseñar un algoritmo que le permita a Diego organizar su agenda para maximizar el score de interés total entre todas las charlas a las que asistirá.

2.8) Convención 3 - Charlas cortables

Para CDEA 2028, encontraron un centro cultural buena onda que permite a los asistentes entrar y salir de las charlas en cualquier momento. Al escuchar esto, Camila decidió mejorar su algoritmo planificador para poder asistir a más charlas. Como le interesa conocer un poco de cada tema, no le importa llegar tarde o irse antes, siempre y cuando pueda asistir a por lo menos el 50% de la charla. Modificar el algoritmo para que Camila pueda organizar su agenda para maximizar la cantidad de charlas a las que asiste.

2.9) Convención 4 - Charlas cortables (cont.)

Diego también asistirá a CDEA 2028, y quiere mejorar su agenda teniendo las nuevas políticas en cuenta. Como entra rápido en el tema, le parece bien entrar a una charla en cualquier momento antes de la mitad, pero como le gusta hacer preguntas a los oradores, quiere quedarse hasta el final en cada charla a la que asiste. Modificar el algoritmo para que Diego pueda organizar su agenda teniendo en cuenta estas restricciones. Puede considerar que todas las charlas le parecen igual de interesantes a Diego (el lineup de este año está mucho mejor).

2.10) Arquitecto 1 - Cobertura de puertas

Armando, el arquitecto, fue contratado por la ciudad para diseñar un nuevo aeropuerto internacional con una capacidad inigualable para mover pasajeros. Ya tiene diseñado el plan de piso de la terminal A, que tiene forma de un largo pasillo recto con puertas de abordaje de un lado y locales de Duty Free del otro. Quiere colocar puestos para el personal de seguridad en algunas de las puertas, de tal modo que ninguna puerta quede a una

distancia mayor de d metros de algún puesto de seguridad. Diseñar un algoritmo que, dado un arreglo con la ubicación de cada puerta (representada en metros desde el inicio de la terminal), le permita a Armando asignar la menor cantidad de puestos de seguridad posible.

2.11) Arquitecto 2 - Cobertura de puertas (cont.)

Armando diseñó ahora la terminal B del aeropuerto, pero por temas de espacio, no puede colocar los puestos de seguridad al lado de las puertas. Para resolver el problema, decidió utilizar algunos de los locales del Duty Free que quedan por rentar como puestos de seguridad. Diseñar un algoritmo que, dado un arreglo con la ubicación de las puertas y otro arreglo con la ubicación de locales de Duty Free libres, le permita a Armando asignar la menor cantidad de puestos de seguridad posible para cubrir todas las puertas.

2.12) Arquitecto 3 - Stands separados (discreto)

Tras su éxito en el diseño aeroportuario, a Armando lo contrataron de nuevo de la ciudad, esta vez para mejorar sus parques y espacios verdes. En el Parque El Rectángulo, que tiene forma de tablero de $n \times m$, quieren colocar puestos de comida en algunas de las casillas. Armando tiene un mapa del parque con las casillas donde pueden ir stands de comida marcadas, pero, para evitar conflictos entre los vendedores, le pidieron que no coloque 2 stands a menos de dos casillas de distancia uno del otro (para este propósito, cuentan las casillas a dos de distancia “en diagonal”). Diseñar un algoritmo que le permita a Armando asignar la mayor cantidad de puestos posibles.

2.13) Arquitecto 4 - Stands separados (continuo)

Le pidieron a Armando que también ayude a asignar los stands de comida en Plaza Euclides. A diferencia de El Rectángulo, esta plaza tiene una forma muy rara para ser descrita como tablero, así que los lugares posibles para stands vienen dados como coordenadas (x, y), y la restricción es que no haya dos stands a menos de 5 metros uno del otro. Diseñar un algoritmo que ayude a Armando a asignar los stands en la plaza.

2.14) Arquitecto 5 - Postas

Previo al verano, a Armando lo contrataron de una ciudad costera para ayudar con la planificación en “El Paseo del Sol”. La ciudad identificó ciertos lugares idóneos para colocar “postas de hidratación” durante las olas de calor para ofrecer agua fresca y tener personal de primeros auxilios. También identificó ciertos puntos críticos donde los turistas suelen pasar más tiempo bajo el sol (p. ej. lugares comunes de fotos, comercios populares que suelen tener fila larga, etc.). Diseñar un algoritmo que, dadas las coordenadas de los lugares para las postas y los puntos críticos, permita poner la menor cantidad de postas posibles de tal modo que ningún punto crítico quede a menos de cierta distancia d de una posta.

2.15) Manager 1 - Guardia mínima

Reggie, el manager del resort Palm Woods, está trabajando en la asignación de horarios. El personal del spa se organiza así: cada uno tiene un turno fijo en la semana en donde trabaja de corrido determinada cantidad de horas. Algunos empleados se les superponen las horas con otros de modo que, dependiendo del día y la hora, hay más o menos empleados en el spa. Reggie quiere mantener una guardia mínima en el spa durante la temporada baja, pero siempre asegurando que haya al menos un empleado en el spa en todo momento para garantizar la experiencia Palm Woods. Diseñar un algoritmo que le permita a Reggie determinar el conjunto mínimo de trabajadores que mantener durante la temporada.

2.16) Manager 2 - Guardia mínima (cont.)

Ahora Reggie se encuentra revisando los puestos del bar de la pileta. Los meseros del bar también tienen horarios fijos, pero pueden venir más de un día a la semana (p. ej. un empleado puede venir los lunes y miércoles de 8 a 14 y los jueves de 12 a 18). Modificar el algoritmo anterior para que Reggie pueda encontrar un subconjunto de meseros que le permita mantener una guardia mínima en el bar minimizando la cantidad de meseros durante la temporada.

2.17) Manager 3 - Supervisores

Ya en vistas a la siguiente temporada, Reggie quiere promover a algunos empleados de la recepción a “manager asistente” para que lo ayuden a supervisar al resto de los empleados. Cada empleado de recepción trabaja un turno durante la semana. Para garantizar el mejor servicio Palm Woods, Reggie quiere asegurarse que todos los empleados de la recepción compartan aunque sea una fracción de su turno con al menos un manager asistente. Diseñar un algoritmo que le permita a Reggie lograr este objetivo promoviendo a la menor cantidad de empleados posible.

2.18) Cambio 1 - Monedas

En el reino de Pharloom, el intercambio económico se hace a base de cuentas y brazaletes de rosario. Existen cuentas/brazaletes con valor 1, 5, y 10, 20, 50, y 120. Jubilana quiere optimizar procesos en su negocio por lo que está diseñando una caja registradora, pero está teniendo problemas con el cálculo del cambio. Diseñar un algoritmo que, dado el valor de los ítems a vender y la cantidad total de cuentas de cada tipo recibidas como pago, calcule cuántas cuentas de cada tipo le debe de cambio al cliente, tratando de minimizar la cantidad de cuentas totales.

2.19) Cambio 2 - Trueque

Jubilana ahora quiere aceptar trueque por distintos tesoros que le traigan aventureros de los confines de Pharloom. Modificar el algoritmo anterior para que, en vez de recibir la cantidad de cuentas de cada tipo dadas como pago, reciba el valor total de cuentas y/o ítems que el

cliente ofreció como pago (que puede ser cualquier valor entero positivo arbitrario), y devuelva el vuelto a dar.

2.20) Cambio 3 - Reforma monetaria (bonus)

Tras ver los problemas que trajo el diseñar un algoritmo para calcular vueltos, los habitantes de Pharloom decidieron que era hora de una reforma monetaria. Oyeron que en el reino vecino de Verdania tenían un sistema basado en hojas de lirio místico, que tenía la particularidad de que, debido a los valores que tenía cada hoja, el algoritmo greedy para calcular vuelto devolvía siempre el vuelto más eficiente en cuanto a cantidad de hojas. Lamentablemente, este sistema quedó en el olvido junto con el reino de Verdania. Encontrar una base monetaria que tenga esta propiedad y demostrarlo.

Capítulo 3 - Backtracking

3.1) Suma de Dados

Implementar un algoritmo de backtracking que reciba una cantidad de dados n y una suma s que devuelva todas las tiradas posibles de n dados cuya suma es s.

Ejemplo:

- Supongamos que recibimos como entrada $n=2$ y $s=5$
- El algoritmo podría devolver *[[1, 4], [2, 3], [3, 2], [4, 1]]* (no importa el orden).

3.2) Maximizar sumatoria a N

Dada una lista de enteros positivos L y un entero n, implementar un algoritmo de backtracking que devuelva un subconjunto de L que sume exactamente n, o, en caso de no existir, que devuelva el subconjunto de suma máxima sin superar el valor de n.

Ejemplo con subconjunto que suma exactamente n:

- Supongamos que recibimos como entrada $L=[1, 2, 4, 6]$ y $n=10$
- El algoritmo deberá devolver *[4, 6]* ya que no hay otro subconjunto que sume exactamente 10.

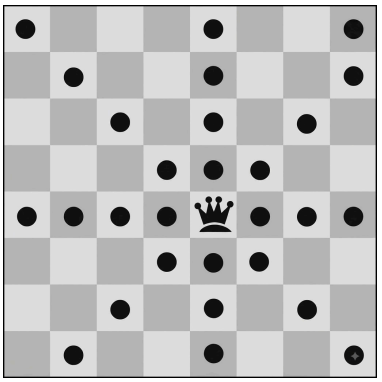
Ejemplo con suma directa no existente por lo que se busca maximizar la suma:

- Supongamos que recibimos como entrada $L=[3, 5, 8, 12]$ y $n=14$
- El algoritmo deberá devolver *[5, 8]* que suma 13 (máxima suma posible utilizando los elementos del conjunto L).

3.3) N Reinas

Dado un tablero de ajedrez $n \times n$, implementar un algoritmo de backtracking que ubique exactamente (si es posible) a n reinas de tal manera que ninguna pueda comerse con ninguna.

Ejemplo visual del movimiento de una reina en el tablero:



- Se desplaza en horizontal, vertical y diagonal.
- Puede avanzar cualquier número de casillas libres en una sola jugada.

3.4) Sudoku

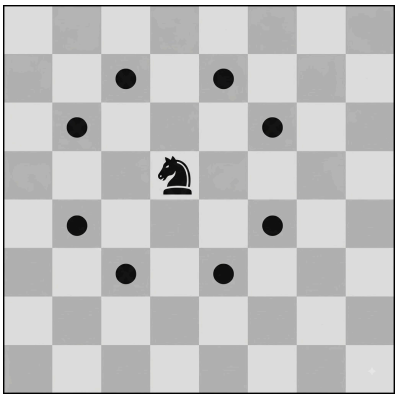
Dada una matriz de $n \; x \; n$, implementar un algoritmo de backtracking que llene la matriz con números del 1 al 9, dadas las condiciones del Sudoku (si es posible). Las condiciones son:

i. Las celdas están dispuestas en 9 subgrupos de $3 \; x \; 3$.
ii. Cada columna y cada fila no puede repetir número.
iii. Cada subgrupo de $3 \; x \; 3$ no puede repetir número.

3.5) Knight Tour

Implementar un algoritmo de backtracking que, dado una pieza de caballo en un tablero de ajedrez de $8 \; x \; 8$, determine los movimientos a realizar para que el caballo logre pasar por todos los casilleros del tablero una única vez.

Ejemplo visual del movimiento del caballo en el tablero:



- Dos casilleros en una dirección, y un casillero en forma perpendicular.
- Se mueve en forma de L.

3.6) Coloreo de Grafos

Implementar un algoritmo que reciba un grafo y un número n que, utilizando backtracking, indique si es posible pintar cada vértice con n colores, de tal forma que no haya dos vértices adyacentes con el mismo color.

3.7) Camino Hamiltoniano

Un camino hamiltoniano es un camino de un grafo que visita todos los vértices del grafo una sola vez. Implementar un algoritmo de backtracking que encuentre un camino hamiltoniano de un grafo dado.

3.8) Maximum Independent Set

Implementar un algoritmo que, dado un grafo no dirigido, nos devuelva un conjunto de vértices que representen un Maximum Independent Set del mismo. Recordemos que un Independent Set es un subconjunto de vértices del grafo en el cual ningún par de ellos es adyacente entre sí.

3.9) Isomorfismo de Grafos

Implementar un algoritmo de backtracking que, dados dos grafos, determine si existe un Isomorfismo entre ambos. Recordemos que dos grafos son isomorfos si existe una correspondencia biyectiva entre sus vértices que preserva las adyacencias; en otras palabras, si estructuralmente son exactamente el mismo grafo aunque estén "dibujados" distinto.

3.10) Cuadrados de Lagrange

El teorema de los cuadrados de Lagrange especifica que cualquier número entero no negativo n puede expresarse como la suma de cuatro cuadrados de enteros no negativos: $n = w^2 + x^2 + y^2 + z^2$. Implementar un algoritmo de backtracking que, dado el valor de n, indique cuales son los cuadrados que lo componen. ¿Cuál es su complejidad?
Ejemplos:

- $42 = 4^2 + 4^2 + 3^2 + 1^2$
- $310 = 17^2 + 4^2 + 2^2 + 1^2$

3.11) SEND + MORE = MONEY

Para el problema de fuerza bruta “SEND + MORE = MONEY” que vimos en la primera clase (8 ciclos anidados de 0 a 9), indicar qué podas realizarías para agilizar su resolución por backtracking. **No se pide pseudocódigo**, sólo explicar las podas.